Title
STAT 542 Final Project

Ajith Senthil, Jerry Hu, Ethan Torres, Dhruv Oza, Cade Duckworth

1 Introduction

Current development of methods used for machine learning tasks associated with natural language processing (NLP) is of popular interest and occurring at increasingly rapid rates. Considering the scope of this unsupervised learning project, we acknowledge that there exist more sufficient semi-supervised machine learning workflows which include fine-tuned and pre-trained layers of various methods from transformer facilitated embedding/tokenization/vectorization, to naive bayes classifiers, and convolutional neural networks, which are better suited to carry out tasks such as NLP, semantic analysis, and document classification. With that said, it is important to note that our workflow for implementing the selected clustering algorithms is untrained and unsupervised from start to finish. Since our dataset is annotated with tweet emotion labels, to gain some insight to the efficacy of our clustering results we have added a supervised post-learning prototyping layer to evaluate the extent to which the clustering results represent the predetermined semantics of the tweets. This required splitting the initial dataset into training and testing subsets, at a ratio of 1:4, respectively. No training information was used for text embedding, dimensionality reduction, or clustering methods, so we expect that partitioning the dataset and adding a prototyping layer for analysis should not introduce bias into the results of the unsupervised learning tasks (K-means and GMM clustering). A more detailed discussion follows in several sections throughout the report. We also provide the links to our google colab:

`https://colab.research.google.com/drive/105pnhaNpM2MwBQgB-aHVRkmfVdXxTOGh?usp=sharing`
`https://colab.research.google.com/drive/1rBn39K4HJFLI-H5Y-hNwcXe_XFXiJ8Of#scrollTo=GAfp_S4tWdOy`
`https://colab.research.google.com/drive/1no8NAAMy6suEi1WdECSVe1MHNb-wL8Mq#scrollTo=vKN2j9awlnVi`

1.1 Machine learning task

Sentiment analysis, often known as opinion mining, is a method for detecting whether an author's or user's viewpoint on a subject is positive or negative [1]. Emotional sentiment analysis is defined as the process of obtaining meaningful information and semantics from text using natural processing techniques and determining the writer's attitude, which might be positive, negative, or neutral. [1].

In this project, we employ the "emotions" dataset to perform a machine learning task where the primary goal is to cluster treets based on the emotions they express. Before clustering, we aim to perform data preprocessing, which involves data cleaning, data vectorization, data embedding, and dimensionality reduction by LSA, without the usage of any pre-trained models. We then attempted to evaluate these methods via various metrics and more involved methods that will be described later in our report.

1.2 Project goals

The primary goal of this project is to classifying the treets based on the emotions they express by applying two distinct clustering methods: Kmean and Gaussian mixture model(GMM). Our aim is to rigorously compare these two methods to determine which clustering method is more efficient and suitable at classifying the treets based on their different emotions. These comparisons will help identify the most suitable approach for our emotional sentiments analysis.

1.3 Project purpose

The purpose of the project is to improve the understanding of sentiments analysis by utilization different advanced clustering methods on the "emotions" dataset. The project aims to explore the effectiveness of these clustering methods in categorizing textual data by implementing both the Kmeans and Gaussian mixture model. Through the comparison of both clusters, we intend to discover the capacities and limitations of Kmeans and Gaussian mixture model in classifying treets that express different emotions.

1.4 Dataset: emotions

The "Emotions" dataset is a comprehensive collection of treets in English, with each tweet coupled with one of the six fundamental emotions: sadness, joy, love, anger, fear, and surprise. The emotions are encoded as: 0 for sadness, 1 for joy, 2 for love, 3 for anger, 4 for fear, and 5 for surprise. There are two columns existing in the original dataset, with one column containing the collection of tweets, and the other column containing the corresponding encoded emotions.

The Emotions dataset was annotated manually (this is crucial in understanding accuracy in terms of actual implementation) with six emotions: anger, fear, joy, love, sadness, and surprise. These each have been indexed with values $0 \rightarrow 5$ which corresponds to their classification label. It should be noted that this dataset is quite large with about $400,000$ unique tweets with $121,000$ sadness (0), $141,000$ joy (1), $35,000$ love (2), $57,000$ anger (3), $47,000$ fear (4), and $15,000$ surprise (5).

This dataset serves as a valuable resources for exploring and analyzing various emotions expressed in the form of texts. With over 400,000 observations of treets, the dataset provides a robust basis for sentiment analysis. Clustering methods, such as Kmean and HDBSCAN, are being employed to identify different groups of treets according to their emotional content. Through the analysis of these clusters and groups, we are able to create a versatile prototype that is capable of accurately capturing the emotions expressed by treets or texts on social media. [2]

2 Algorithms & Methods

2.1 Underlying mathematical concepts of the algorithms

2.1.1 K-means clustering

First, we want to describe the underlying spaces the exist. For one, we have an observation space that we will denote by $\mathbf{x} = (x_1, x_2, ..., x_n)$. Ideally, we seek to minimize the variance and optimally solve the objective function:

$$\arg\min_{\mathbf{C}} \sum_{i}^{k} \frac{1}{|C_i|} \sum_{x_i, x_j \in C_i} ||x_i - x_j||^2 \tag{1}$$

where C is a set, the cluster, where each cluster is has $k \leq n$ clusters with $\mathbf{C} = \{C_1, C_2, ..., C_k | C_i \cap C_j = \emptyset \ \in \mathbb{R}^2\}$ i.e. they are pairwise disjoint clusters. This is clear given the motivation of the problem is too classify text information into emotional categories that should not mix. Furthermore, we point out that $||x_i - x_j|| \in L^2$ to be clear that we are trying to classify points given a standard Euclidean distance in two dimensions. Below I will show a sample of how our algorithm attempts to use the equation shown above to implement the method:

```
initialize_centroids(k):
    randomly select k data points as initial centroids

assign_to_clusters(data, centroids):
    for each data point in data:
        find the nearest centroid
        assign the data point to the cluster of the nearest centroid

update_centroids(data, centroids):
    for each centroid:
    calculate the mean of the data points assigned to the cluster
    update the centroid to be the mean

kmeans(data, k):
    centroids = initialize_centroids(k)
    prev_centroids = None
    while centroids are not converging:
        prev_centroids = centroids
        assign_to_clusters(data, centroids)
        update_centroids(data, centroids)
    return cluster assignments and centroids
```

Here note that the above equation we are attempting to minimize is a mathematical equivalent to the statement:

$$\arg\min_{C} \sum_{i=1}^{k} \sum_{x \in C_i} ||\mathbf{x} - \mu_\mathbf{i}||^2$$

where we have μ_i to be the mean of the centroid (which is what the pseudocode is attempting to calculate):

$$\mu_\mathbf{i} = \frac{1}{|C_i|} \sum_{x \in C_i} \mathbf{x}$$

The reason we include equation (1) is because it more accurately describes the topological motivation behind the algorithm, which is to take a point and place it in $\mathbb{R}^2$ and then find the cluster that is its nearest neighbor.

2.1.2 Gaussian mixture model (GMM)

What do we seek to achieve with the Gaussian mixture model that is different than k-means clustering? The core underlying principle is probability. Unlike k-means clustering, which is entirely deterministic, the GMM introduces

probabilistic assumptions to attempt to improve accuracy in clustering. [3] Once again we start with a set of observations $\mathbf{x} = (x_1, x_2, ..., x_n)$. We then introduce the probabilistic aspect given by a multivariate guassian distribution:

$$\mathcal{N}(x_i, \mu_C, \Sigma_C) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma_C|^{\frac{1}{2}}} e^{-\frac{1}{2}(x_i - \mu_C)^T \Sigma_C^{-1} (x_i - \mu_C)}$$

Then, we calculate at each step of the algorithm, whether or not our data point actually belongs to $\mathbf{C} = \{C_1, C_2, ..., C_k | C_i \cap C_j = \emptyset\}$. We do this by applying Bayes theorem:

$$r_{iC} = \frac{\pi_C \mathcal{N}(x_i | \mu_C, \Sigma_C)}{\sum_{k=1}^{K} \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}$$

where π_C is our weight or mixing coefficient. The mixing. Generally, this is the likelihood that a particular data point belongs to each cluster. When the algorithm is initialized, these probabilities are equal and have equal probability distribution implying that the weight parameter is also equal for each component i.e. $\mathcal{N}(a, b, c)$ where $a = b = c$. As the algorithm updates at each step, this probability changes. It is also important to note that μ and Σ - the mean and covariance matrix of the set respectively - is initialized randomly with some random seed value. As the algorithm progresses, all three of these parameters are updated using the following rules:

$$\pi_C = \frac{\sum_{i=1}^{m} r_{iC}}{m} \tag{2}$$

$$\mu_C = \frac{\sum_{i=1}^{m} r_{iC} x_i}{\sum_{i=1}^{m}} \tag{3}$$

$$\Sigma_C = \frac{\sum_{i=1}^{m} r_{iC} (x_i - \mu_C)^2}{\sum_{i=1}^{m} r_{iC}} \tag{4}$$

where we have taken m to be the number of data points. After updating each of these parameters, we seek to compute the log-likelihood of the observed data which is given by:

$$\ln p(X|\theta) = \sum_{n=1}^{N} \ln \sum_{k=1}^{K} \pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k)$$

Ultimately, we seek to maximize the log-likelihood from this algorithm. The algorithm is determined by us to be completed when this has run through a certain amount of iterations and this values is below some threshold bound. From here, we determine convergence. [4]

2.2 Methods for algorithm implementation

2.2.1 Data pre-processing

Data preprocessing is one of the most important steps to properly analyzing textual information, especially information like the emotions dataset. For one, tweets are very informal and tend to have lots of abbreviations and colloquial slang in them. In addition, there are words that are typically omitted, or included, that don't belong and would potentially confuse the algorithm. As a result, we utilize "nltk" which is the Natural Language Toolkit package. Then, we loop through the entire data set's textual column and apply what are called stopwords in the nltk package. This is just one example of a filter included that is extremely useful for textual analysis. In addition, we purge the dataset of any duplicates and then we map the dataset so we are able to cleanly visualize the dataset.

Ultimately, the reason we cleaned at all was to achieve data consistency and try to improve accuracy of the analysis. For one, terms like 'lol' and 'bff' can be easily misinterpreted by an algorithm when it comes to determining the emotional set that each tweet belongs too. For two, the dataset itself is extremely long and when it comes to huge datasets, there are frequently inconsistencies and small errors that occur. Cleaning the mess, so to speak, allows the dataset to be as "pure" as it possibly can be which will ultimately achieve the best results. Moreover, it is much easier to create and call functions later when there is a level of consistency in the dataset. [5]

2.3 Splitting the dataset

In this part of the project, the dataset is split into training and testing sets for future evaluation, analysis, and validation purposes by employing the the prototyping classification methods to compare Kmean and GMM clusters. The training set and testing set are created merely for the purpose of evaluation, rather than the performance.

Term frequency-inverse document frequency text vectorization (tf-idf) method is employed to convert text data into their numerical forms to signify the importance of the variety of terms within the dataset. Subsequently, A

Latent Semantic Analysis (LSA) is applied to the dataset for the purpose of dimensionality reduction to improve the readability and interpretability of the text data while maintaining the key features of the data.

For Kmeans cluster, we start by specifying the number of clusters that is needed. In this case, there are six fundamental emotions, and therefore, K=6. Six data points were randomly assigned as the centroids, in which the algorithm is ran in R repeatedly until the six points are stabilized as the centroids [6]. Subsequently, we would compute the sum of the squared distance between data points and all centroids and assign each data point to the closest cluster [6].

The Gaussian Mixture Model (GMM) clustering algorithm assigns data points to clusters according to their individual probabilities of belonging to the clusters. GMM would assume that the data is generated from different Gaussian distributions, in which each distribution represents a cluster. In this case, there would be six clusters and accordingly six different Gaussian distribution. Data points will be assigned probabilities based on their distance from the center of the clusters and the spread of cluster during the clustering process. Subsequently, the data points with the highest probabilities will be selected and classified into the clusters.

3 Results

3.1 K-means

3.1.1 Metrics

```
clustering done in 3.47 ± 1.78 s
Homogeneity: 0.002 ± 0.001
Completeness: 0.003 ± 0.001
V-measure: 0.003 ± 0.001
Adjusted Rand-Index: 0.001 ± 0.001
Silhouette Coefficient: 0.055 ± 0.003
```

Figure 1: Kmeans Metrics

3.1.2 Results

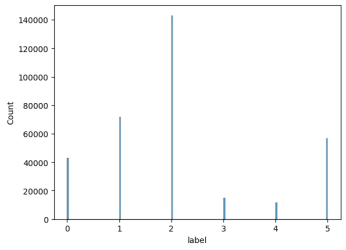

Figure 2: K-Means Metrics

3.2 GMM

3.2.1 Metrics

```
clustering done in 362.74 ± 47.27 s
Homogeneity: 0.006 ± 0.002
Completeness: 0.005 ± 0.002
V-measure: 0.006 ± 0.002
Adjusted Rand-Index: 0.004 ± 0.002
Silhouette Coefficient: 0.008 ± 0.001
```

Figure 3: GMM Metrics

3.2.2 Results

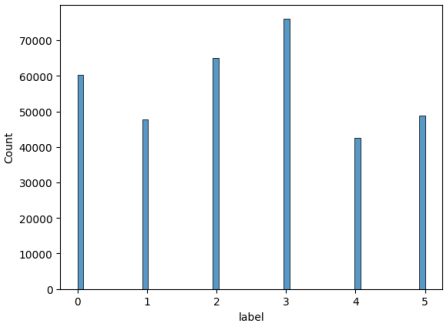

Figure 4: GMM Cluster Results

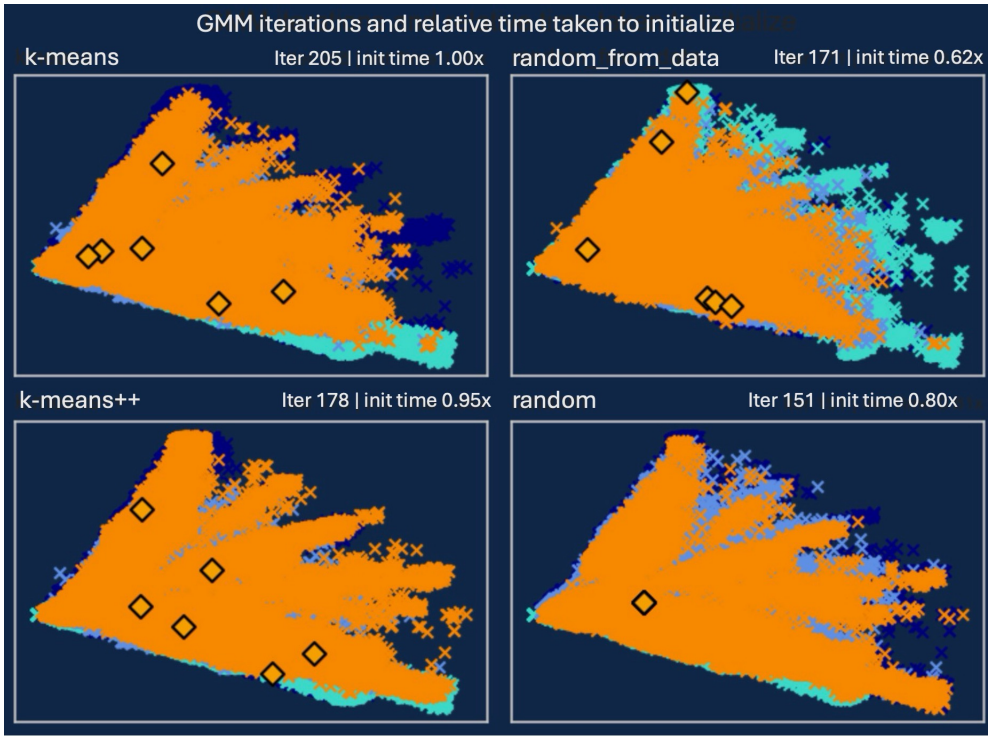

Figure 5: GMM Results w/ 4 Init Methods

3.3 Prototyping

3.3.1 Comparison

In this study, we compared two approaches to classifying tweet sentiments: using predefined labels (ground truth) and using k-means clustering to generate labels.

Ground Truth Classification We started with tweets that already had labels indicating their emotional content. We embedded these tweets using a Sentence Transformer model, calculated the centroids for each emotion category, and identified the tweets closest to these centroids. Here are some examples:

- Cluster 0: "I feel depressed and disappointed and disillusioned"
- Cluster 1: "I feel reassured and motivated"
- Cluster 2: "I feel really beloved and blissful"
- Cluster 3: "I feel irritated and I kinda hate this feeling"
- Cluster 4: "I confess I'm feeling more than a little anxious"
- Cluster 5: "I have spent feeling shocked or sad I have also felt a little bit of joy"

K-means Clustering For the k-means approach, we did not use pre-assigned labels. Instead, we clustered the tweets based on their embeddings and then found the tweets closest to each cluster's centroid. Examples include:

- Cluster 2: "I was feeling really depressed and sensitive which is not out of character then I could be more appreciative of this"
- Cluster 5: "I feel disheartened and frustrated by the experience"
- Cluster 3: "I am feeling genuinely hopeful and kind of happy"
- Cluster 4: "I feeling so distressed"

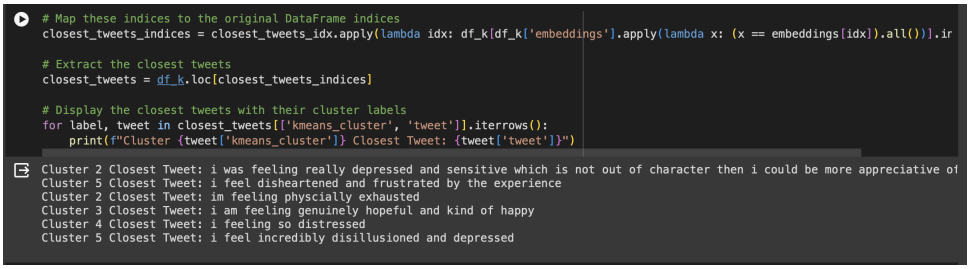

Figure 6: Closest Tweets for K-Means Clusters

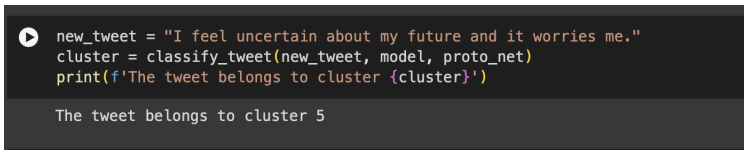

Figure 7: Example of Prototype Classification

Analysis Comparing these approaches shows how k-means clustering can sometimes reflect predefined emotions but can also offer different interpretations. This suggests that unsupervised clustering can capture nuances that predefined categories might miss.

We also evaluated the clustering effectiveness using metrics like homogeneity, completeness, and silhouette scores. These measurements help us understand how well the clusters formed around the prototypes align with the actual data groupings.

The figure above (3) illustrates the performance of each clustering method, providing a clear visual representation of their effectiveness in capturing the emotional content of tweets.

3.4 Utilizing Prototypes for Ranking Metrics

This section outlines the method of using prototype-based direction vectors to create ranking metrics. This technique involves computing directional vectors from the embeddings of the most and least representative tweets within a cluster, and subsequently using these vectors to rank other tweets based on their semantic similarity or emotional alignment.

3.4.1 Directional Vector Calculation

Direction vectors are calculated between the embeddings of the most and least representative tweets of a cluster. This vector represents the semantic or emotional transition from one state to another within the cluster context. For each cluster, the following steps were performed:

1. Identify the most and least representative tweets based on their proximity to the cluster centroid.
2. Compute the embedding for each tweet.
3. Subtract the embedding of the least representative tweet from the most representative tweet to form the directional vector.

3.4.2 Ranking Tweets Using Direction Vectors

With the directional vectors established, we can project other tweets onto these vectors to measure their alignment with the defined semantic or emotional trajectory. This method ranks tweets by their projection magnitudes, indicating how closely each tweet aligns with the emotional or semantic path defined by the vector. The procedure is as follows:

1. For each tweet in the dataset, calculate the dot product of its embedding with the direction vector.
2. Normalize this value by dividing by the norm of the direction vector to get the projection magnitude.
3. Rank all tweets based on these projection magnitudes, with higher values indicating closer alignment with the directional vector.

3.4.3 Implementation Example

Consider a cluster where the most representative tweet expresses a strong positive emotion and the least representative tweet expresses a strong negative emotion. The directional vector from the negative to the positive tweet defines an emotional trajectory from negative to positive. By projecting other tweets onto this vector, we can rank them according to their positivity.

```
# Calculate the direction vector
direction_vector = np.array(most_rep['embedding']) - np.array(least_rep['embedding'])

# Function to project and rank tweets
def project_tweet(tweet_embedding):
    norm_vector = np.linalg.norm(direction_vector)
    return np.dot(tweet_embedding, direction_vector) / norm_vector

# Apply to all tweets
df['projection'] = df['embeddings'].apply(project_tweet)
ranked_tweets = df.sort_values(by='projection', ascending=False)
```

This approach allows us to quantify and rank the tweets according to their semantic or emotional content relative to the prototypical emotional journey defined within each cluster. This can be particularly useful for understanding complex emotional dynamics in large-scale text data, such as social media feeds or customer feedback systems.

3.5 Mathematical analysis and comparison of results

There were multiple clustering assessment metrics that we checked so that we would be able to compare basic results on the efficiency and quality of the two algorithms. These metrics included clustering time, homogeneity, completeness, V-measure, adjusted rand-index, and silhouette coefficient. In the above figures it is clear that GMM is, and should be, a much better algorithm in general when it comes to textual, and by extension of course sentiment, analysis. First, time. K-means we know to be faster because it quickly partitions a dataset into clusters and then easily sorts them. This is clearly true as it is nearly 100 times faster than GMM. However, due to the complicated nature of the data and the way in which it is distributed, we expected GMM (and the math clearly alludes to it as well) to perform much better. This is the case for homogeneity, completeness, V-measure, and ARI. Homogeneity, which is given by:

$$h = 1 - \frac{H(C|K)}{H(C)}$$

where

$$H(C|K) = -\sum_{c,k} \frac{n_{ck}}{N} \log\left(\frac{n_{ck}}{n_k}\right)$$

tells us how similar samples in a cluster happen to be for a given algorithm. Clearly, the above term $H(C|K)$, which shows the ratio between the samples in cluster c in cluster k over the total number of samples in k, tells us how much one cluster has values that exist in another cluster. This is a really good metric for showing the accuracy of a dataset especially one like the emotions dataset where emotions can be similar but are, for the most part, sorted specifically into explicit categories. Clearly, due to the complicated nature of the dataset and nonlinearity of the dataset it is clear that GMM should have higher homogeneity - and it does. Completeness tells us a similar story where a higher

completeness score indicates that data points belonging to the same class are assigned to the same cluster. This is given by:

$$c = 1 - \frac{H(K|C)}{H(K)}$$

Where once again $H(K|C)$ provides a ratio which contains the number of samples labelled in c in k to the total number of samples in c. This, once again, clearly tells us that GMM should have greater completeness score as it can deal with much more complex datasets with more complex shapes and distributions. Given the higher scores for both completeness and homogeneity, it is clear that GMM should have a higher V-measure as well, which is simply the harmonic mean of completeness and homogeneity. [7] Clearly our results show this. Adjusted rand-index is a similar metric that measure the similarity between two clusters, but it provides an adjustment for randomness. It does this by measuring the agreement between the clusters produced and the ground truth labels correcting for the expected agreement due to chance. This is given by the well known formula in [Hubert and Arabie, 1985]:

$$\frac{\sum_{i,j}\binom{n_{ij}}{2} - \left[\sum_i \binom{n_{i.}}{2}\sum_j \binom{n_{.j}}{2}\right]/\binom{n}{2}}{\frac{1}{2}\left[\sum_i \binom{n_{i.}}{2} + \sum_j \binom{n_{.j}}{2}\right] - \left[\sum_i \binom{n_{i.}}{2}\sum_j \binom{n_{.j}}{2}\right]/\binom{n}{2}}$$

While it is hard to determine necessarily which should have a better ARI before seeing the results, intuitively given the dataset and the well defined nature of the clusters themselves, it stands to reason that given an adjustment for randomness, a probabilistic algorithm which incorporates this randomness to achieve better results would have a better ARI when this randomness is taken into account. This also goes hand in hand with the nature of the dataset. After running the algorithms and running the metrics, this statement is clearly verified with GMM have a ARI that is 4 times greater. The only anomaly that existed was in our silhouette coefficient. It is obvious that GMM should have a larger coefficient. However, there seems to have been some error that existed in our calculation or in our dataset but we were not able to properly diagnose it.

4 Discussion & Difficulties

4.1 Difficulties

4.1.1 Trials with HDBScan

HDBScan allows changing parameters such as minimum cluster size, cutoff distances for merging clusters, and several others to control the results of the clustering. After trying multiple combinations of parameters for HDBScan, we found that attempting to coerce the clustering algorithm into producing six clusters was difficult and finding a way to do so would waste considerable amounts of time. Additionally, in many cases it seemed like a significant amount of tweets were labelled as noise by HDBScan and discarded. This behavior did not align with our project goals, so we decided to pursue another algorithm, at which point we decided to pursue use of GMM instead.

4.1.2 Parameter selection for K-means and GMM

Parameters for K-means included initialization, number of iterations, and number of clusters. Parameters for GMM were more complicated. We tested different initialization methods running GMM, including kmeans, kmeans++, random_from_data, and random, with results shown in Figure 3. For validity of comparison of both algorithms with the input dataset, we specified the number of clusters as 6 for both algorithms. We opted for standard parameters that were shown in Scikit-Learn examples to form a base comparison. If more time were allowed, we would set up an iterative workflow to compare the performance of the two algorithms under many varied sets of parameter combinations.

4.1.3 Prototype Classes

Due to the time constraints, we were unable to run the few shot classification for the prototypical networks, and we were not able to run a proper comparison on our newest clustering method results. We wanted to run it on a large test set and compare accuracies and make a confusion matrix to compare with other prototype classes derived from our other clustering methods and also known supervised learning techniques.

4.2 Discussion

4.2.1 Conclusions from results

It is clear that despite the difficulties that exist between our implementations of GMM and K-Means and the complicated nature of our dataset, both algorithms have their validity when it comes to sentiment analysis. While it

is difficult to choose a clear "winner", GMM seems to generally be a much more powerful tool for complex sentiment analysis than K-Means. K-Means, however, is still a very powerful tool for getting accurate results in a much better time-frame. Moreover, given the multitude of possible advancements and tweaks than can be done for both algorithms such as prototyping and HBDscan, it is clear that each algorithm has the potential to perform these high level tasks and play to each of their respective strengths given certain tasks.

References

[1] Nandwani P and Verma R. A review on sentiment analysis and emotion detection from text. *Advances in Neural Information Processing Systems*, page 2, 2021.

[2] Nelgiri Withana. Emotions dataset. Kaggle, 2021. https://www.kaggle.com/datasets/nelgiriyewithana/emotions?resource=download.

[3] scikit-learn contributors. scikit-learn: Gaussian mixture models. scikit-learn Documentation, Year Accessed. https://scikit-learn.org/stable/modules/mixture.html.

[4] Data Science Stack Exchange Community. Math of gaussian mixture models (em algorithm). Data Science Stack Exchange, Year Accessed. https://datascience.stackexchange.com/questions/121086/math-of-gaussian-mixture-models-em-algorithm.

[5] Saman Fatima. Crushing it: Bilstm rnn delivers 94 Kaggle, 2021. https://www.kaggle.com/code/samanfatima7/crushing-it-bilstm-rnn-delivers-94-accuracy.

[6] Author(s). K-means Clustering: Algorithm, Applications, Evaluation Methods, and Drawbacks. https://towardsdatascience.com/k-means-clustering-algorithm-applications-evaluation-methods-and-drawbacks-aa03 2018. /.

[7] Towards Data Science. V-measure: An homogeneous and complete clustering. Towards Data Science, Year Accessed. https://towardsdatascience.com/v-measure-an-homogeneous-and-complete-clustering-ab5b1823d0ad.